WEEK 9

Coroutines & Flow

Daily study material with detailed examples

Android Development with Kotlin | Prepared for **Anjana**

Apps constantly wait: for the network, the database, a timer. If the app waited on the **main thread**, the screen would freeze. **Coroutines** let you write waiting code that looks simple and sequential but never blocks the UI. **Flow** lets you handle values that arrive over time. You've already used both (viewModelScope, StateFlow). This week you'll understand them properly, and build a **Countdown Timer** app.

Day	Topic	You will be able to...
Day 1	Suspend functions & launch	Explain the main thread and write your first coroutines
Day 2	async, Jobs & structured concurrency	Run work in parallel and cancel it safely
Day 3	Dispatchers, scopes & errors	Choose the right thread, handle errors and timeouts
Day 4	Flow basics	Create, transform and collect flows
Day 5	StateFlow, combine & debounce	Build reactive ViewModels, such as live search
Day 6	Weekly project	Build a Countdown Timer app

> **Tip:** The plain Kotlin examples were **run** with the real kotlinx.coroutines library, and the outputs shown are real. Use the Kotlin Playground (it supports coroutines) or a Kotlin file in IntelliJ. Compose code is compile-checked.

WEEK 9 • DAY 1

Suspend Functions & launch

Today's goal: Understand why the main thread must never block, and write coroutines with suspend, launch and delay.

1. The main thread

Android draws the screen and handles taps on one thread, the **main thread** (also called the UI thread). It must redraw about 60 times per second. If you make it wait for a slow network call, the app **freezes**, and after about 5 seconds Android shows the dreaded **"App isn't responding" (ANR)** dialog.

Sketch: Blocking vs suspending

```
Blocking (bad):     main thread  [draw][draw][..... waiting 3s for network .....][draw]
                                                screen frozen, taps ignored

Suspending (good):  main thread  [draw][draw][draw][draw][draw][draw][draw][draw][draw]
                    coroutine          start --- suspended (waiting) --- resume [update UI]
```

A **coroutine** is a piece of work that can **pause** (suspend) while waiting, and **resume** later, without blocking the thread. Thousands of coroutines can share a few threads.

2. Your first coroutine

launch and delay

```
import kotlinx.coroutines.*

fun main() = runBlocking {
    println("1. Start")

    launch {                        // start a new coroutine
        delay(500)                  // pause this coroutine for 500 ms (doesn't block the thread)
        println("3. Done waiting")
    }

    println("2. Main code keeps going")
}
```

Output

```
1. Start
2. Main code keeps going
3. Done waiting
```

Code	Meaning
runBlocking { }	Starts coroutines in a normal main() and waits for them. **Only for learning and tests**, never in Android UI code
launch { }	Starts a new coroutine that runs alongside the current code ("fire and forget")
delay(500)	Suspends the coroutine for 500 ms. Unlike Thread.sleep, the thread is free to do other work

Careful: If the **last line** inside `fun main() = runBlocking { }` is a launch { } or a try/catch, Kotlin says *'main' method not found*, because main then returns that value instead of Unit. Fix it by ending with a normal statement such as println("Done"), or by writing `fun main(): Unit = runBlocking { }`.

3. Suspend functions

A function marked **suspend** can pause. It can only be called from a coroutine or from another suspend function. Network and database calls in Android are suspend functions.

Writing suspend functions

```
import kotlinx.coroutines.*

suspend fun fetchUserName(): String {
    delay(300)              // pretend: network call
    return "Anjana"
}

suspend fun fetchUserCity(): String {
    delay(300)              // pretend: another network call
    return "Kochi"
}

fun main() = runBlocking {
    println("Loading...")
    val name = fetchUserName()          // waits here, without blocking
    val city = fetchUserCity()          // then this runs
    println("$name lives in $city")
}
```

Output

```
Loading...
Anjana lives in Kochi
```

The code reads **top to bottom**, like normal code, even though it waits twice. That's the magic of coroutines: no callbacks.

ERROR: calling a suspend function from normal code

```
fun normalFunction() {
    val name = fetchUserName()   // Error: Suspend function 'fetchUserName' should be called only
}                                // from a coroutine or another suspend function
```

4. Several coroutines at once

Coroutines run concurrently

```
import kotlinx.coroutines.*

fun main() = runBlocking {
    launch {
        repeat(3) { i ->
            println("Download A: ${(i + 1) * 33}%")
            delay(100)
        }
    }
    launch {
        repeat(3) { i ->
            println("Download B: ${(i + 1) * 33}%")
            delay(100)
        }
    }
    println("Both downloads started")
}
```

Output

```
Both downloads started
Download A: 33%
Download B: 33%
Download A: 66%
Download B: 66%
Download A: 99%
Download B: 99%
```

Both coroutines share **one thread**. Each time one calls delay, the other gets a turn. This is called **concurrency**.

5. delay vs Thread.sleep

delay(1000)	Thread.sleep(1000)
Suspends only the coroutine	Blocks the whole thread
Other coroutines keep running	Everything on that thread stops
Safe on the main thread	Freezes the app if used on the main thread
Can be cancelled	Can't be cancelled the same way

6. Where coroutines start in Android

Coroutine scopes in Android (Android only)

```
// In a ViewModel: cancelled automatically when the ViewModel is cleared
viewModelScope.launch {
    val user = repository.getUser()
    _uiState.update { it.copy(user = user) }
}

// In a composable: runs when the composable appears, cancelled when it leaves
LaunchedEffect(Unit) {
    delay(2000)
    onSplashFinished()
}

// In a composable, from a click: a scope tied to the composable
val scope = rememberCoroutineScope()
Button(onClick = { scope.launch { snackbarHostState.showSnackbar("Saved") } }) { Text("Save") }
```

Practice Exercises

1. Write a suspend function **makeTea()** that prints "Boiling water", waits 300 ms, prints "Adding tea", waits 200 ms, and returns "Tea is ready". Call it from runBlocking.

2. Launch 3 coroutines that each wait a different time (300, 100, 200 ms) and then print their name. Predict the order, then run it.

3. Change the delay in exercise 2 to Thread.sleep. What changes in the order, and why?

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What happens if the main thread is blocked for about 5 seconds?

Q2. What does suspend mean?

Q3. What is the difference between delay and Thread.sleep?

Q4. Should you use runBlocking in Android UI code?

Q5. Which scope should a ViewModel use?

WEEK 9 • DAY 2

async, Jobs & Structured Concurrency

Today's goal: Run work in parallel with async/await, control coroutines with Job, and understand structured concurrency.

1. Sequential vs parallel

Sequential: one after another

```
import kotlinx.coroutines.*
import kotlin.system.measureTimeMillis

suspend fun loadProfile(): String { delay(300); return "Profile" }
suspend fun loadPosts(): String { delay(300); return "Posts" }

fun main() = runBlocking {
    val time = measureTimeMillis {
        val profile = loadProfile()
        val posts = loadPosts()
        println("$profile + $posts")
    }
    println("Took about ${time / 100 * 100} ms")
}
```

Output

```
Profile + Posts
Took about 600 ms
```

Parallel: async + await

```
import kotlinx.coroutines.*
import kotlin.system.measureTimeMillis

suspend fun loadProfile(): String { delay(300); return "Profile" }
suspend fun loadPosts(): String { delay(300); return "Posts" }

fun main() = runBlocking {
    val time = measureTimeMillis {
        val profile = async { loadProfile() }     // start both at once
        val posts = async { loadPosts() }
        println("${profile.await()} + ${posts.await()}")   // wait for both results
    }
    println("Took about ${time / 100 * 100} ms")
}
```

Output

```
Profile + Posts
Took about 300 ms
```

	launch	async
Returns	A Job (no result)	A Deferred<T> (a future result)
Get the result with	-	.await()
Use for	"Do this" (save, update UI)	"Calculate this" and use the value

> **Tip:** Use async only when you need **several independent results at the same time**, for example loading a profile and their posts together. For one call, just call the suspend function directly.

2. Job: controlling a coroutine

join and cancel

```
import kotlinx.coroutines.*

fun main() = runBlocking {
    val job = launch {
        repeat(10) { i ->
            println("Working... $i")
            delay(100)
        }
    }
    delay(350)               // let it run a little
    println("Cancelling")
    job.cancel()             // ask it to stop
    job.join()               // wait until it has stopped
    println("Cancelled: ${job.isCancelled}")
}
```

Output

```
Working... 0
Working... 1
Working... 2
Working... 3
Cancelling
Cancelled: true
```

Cancellation is **cooperative**: the coroutine stops at the next suspension point (here, delay). Library suspend functions (network, database, delay) all support cancellation.

3. Structured concurrency

Coroutines are organised in a **tree**. A parent waits for its children, and cancelling the parent cancels all its children. This means no coroutine is ever forgotten and left running. That's **structured concurrency**.

Parent and children

```
import kotlinx.coroutines.*

fun main() = runBlocking {
    val parent = launch {
        launch {
            delay(1000)
            println("Child 1 finished")      // never printed
        }
        launch {
            delay(1000)
            println("Child 2 finished")      // never printed
        }
        println("Parent started two children")
    }
    delay(200)
    parent.cancel()                          // cancels the children too
    parent.join()
    println("Parent and children cancelled")
}
```

Output

```
Parent started two children
Parent and children cancelled
```

Sketch: The coroutine tree in an Android app

```
 viewModelScope            <- cancelled when the ViewModel is cleared (user leaves the screen)
   +-- launch { loadProfile() }
   +-- launch { refresh() }
         +-- async { loadPosts() }
         +-- async { loadFriends() }
 Leaving the screen cancels everything automatically. No leaks, no wasted network calls.
```

4. coroutineScope: parallel work inside a suspend function

Grouping parallel work

```
import kotlinx.coroutines.*

suspend fun loadDashboard(): String = coroutineScope {
    val sales = async { delay(200); 1500 }
    val orders = async { delay(100); 42 }
    "Sales: Rs ${sales.await()}, Orders: ${orders.await()}"
}

fun main() = runBlocking {
    println(loadDashboard())
}
```

Output

```
Sales: Rs 1500, Orders: 42
```

coroutineScope { } creates a small scope inside a suspend function. It returns only when all its children are done, and if one child fails, the others are cancelled.

Practice Exercises

1. Write suspend functions getWeather() (400 ms) and getNews() (300 ms). Load them in parallel with async and print the total time.

2. Start a coroutine that prints a countdown from 10 every 100 ms. Cancel it after 450 ms. Which numbers are printed?

3. Write **suspend fun loadAll(ids: List<Int>): List<String>** that loads each id in parallel (delay(100) then "Item id") using coroutineScope and map { async { } }.awaitAll().

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What is the difference between launch and async?

Q2. Why does the parallel version take about 300 ms instead of 600 ms?

Q3. When does a cancelled coroutine actually stop?

Q4. What happens to children when a parent coroutine is cancelled?

Q5. What is structured concurrency?

WEEK 9 • DAY 3

Dispatchers, Scopes & Errors

Today's goal: Run code on the right thread with dispatchers and withContext, and handle errors, cancellation and timeouts.

1. Dispatchers: which thread runs the code

Dispatcher	Use for	Example
Dispatchers.Main	Updating the UI (Android)	Default for viewModelScope and LaunchedEffect
Dispatchers.IO	Waiting for input/output: network, files, database	Reading a big file
Dispatchers.Default	CPU-heavy work	Sorting 100,000 items, image processing, JSON parsing of huge data

2. withContext: switch threads for one block

Moving heavy work off the main thread

```
import kotlinx.coroutines.*

suspend fun calculatePrimesBelow(limit: Int): Int = withContext(Dispatchers.Default) {
    (2 until limit).count { n -> (2..kotlin.math.sqrt(n.toDouble()).toInt()).none { n % it == 0 } }
}

fun main() = runBlocking {
    println("Calculating...")
    val count = calculatePrimesBelow(100_000)
    println("There are $count primes below 100,000")
}
```

Output

```
Calculating...
There are 9592 primes below 100,000
```

withContext(Dispatchers.Default) { } runs the block on a background thread, waits for it, and returns its result, then the caller continues on its original thread. The caller doesn't need to know which thread was used.

Tip: Libraries like **Retrofit** (Week 10) and **Room** (Week 11) already switch to background threads inside their suspend functions. You can call them safely from viewModelScope without withContext. This idea is called **main-safety**.

3. Handling errors

try/catch around suspend calls

```
import kotlinx.coroutines.*

suspend fun fetchPrice(product: String): Int {
    delay(100)
    if (product == "unknown") throw IllegalStateException("Product not found")
    return 499
}

fun main() = runBlocking {
    for (p in listOf("book", "unknown")) {
        try {
            val price = fetchPrice(p)
            println("$p costs Rs $price")
        } catch (e: IllegalStateException) {
            println("Error for $p: ${e.message}")
        }
    }
}
```

Output

```
book costs Rs 499
Error for unknown: Product not found
```

4. runCatching: a compact alternative

Result with runCatching

```
import kotlinx.coroutines.*

suspend fun login(password: String): String {
    delay(100)
    require(password.length >= 6) { "Password too short" }
    return "token-abc123"
}

fun main() = runBlocking {
    val ok = runCatching { login("kotlin123") }
    val bad = runCatching { login("123") }

    ok.onSuccess { println("Logged in with $it") }
    bad.onFailure { println("Failed: ${it.message}") }
    println(bad.getOrDefault("no token"))
}
```

Output

```
Logged in with token-abc123
Failed: Password too short
no token
```

Careful: Be careful: catching **all** exceptions (catch (e: Exception) or runCatching) also catches **CancellationException**, which coroutines use to stop. In a loop or long-running work, re-throw it, or check **isActive**, so cancellation still works.

5. Timeouts

withTimeout and withTimeoutOrNull

```
import kotlinx.coroutines.*

suspend fun slowServer(): String {
    delay(2000)
    return "data"
}

fun main() = runBlocking {
    val result = withTimeoutOrNull(500) { slowServer() }     // gives null if too slow
    println(result ?: "Server too slow, showing cached data")

    try {
        withTimeout(500) { slowServer() }                    // throws if too slow
    } catch (e: TimeoutCancellationException) {
        println("Timed out")
    }
    println("Done")
}
```

Output

```
Server too slow, showing cached data
Timed out
Done
```

6. In a ViewModel

Errors in viewModelScope

```
class PriceRepository {
    suspend fun getPrice(id: Int): Int {
        delay(500)
        if (id <= 0) throw IllegalArgumentException("Invalid id")
        return id * 100
    }
}

data class PriceUiState(val price: Int? = null, val isLoading: Boolean = false, val error: String? = null)

class PriceViewModel(private val repo: PriceRepository = PriceRepository()) : ViewModel() {
    private val _uiState = MutableStateFlow(PriceUiState())
    val uiState: StateFlow<PriceUiState> = _uiState.asStateFlow()

    fun load(id: Int) {
        viewModelScope.launch {
            _uiState.update { it.copy(isLoading = true, error = null) }
            val result = runCatching { withTimeout(3000) { repo.getPrice(id) } }
            _uiState.update {
                result.fold(
                    onSuccess = { price -> it.copy(price = price, isLoading = false) },
                    onFailure = { e -> it.copy(isLoading = false, error = e.message ?: "Something went wrong") }
                )
            }
        }
    }
}
```

Practice Exercises

1. Write **suspend fun sumOfSquares(n: Int): Long** that runs on Dispatchers.Default and returns the sum of squares from 1 to n. Print it for 1,000,000.

2. Write a suspend function that throws for negative numbers, and call it with 5 and -5 using runCatching. Print the results.

3. Use withTimeoutOrNull(300) around a function that takes 500 ms, and then around one that takes 100 ms. Print both results.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Which dispatcher is for network and database work?

Q2. Which dispatcher is for heavy calculations?

Q3. What does withContext do?

Q4. Why must you be careful when catching all exceptions in coroutines?

Q5. What is the difference between withTimeout and withTimeoutOrNull?

WEEK 9 • DAY 4

Flow Basics

Today's goal: Create flows, transform them with operators, and collect their values.

1. What is a Flow?

A suspend function returns **one** value. A **Flow** returns **many values over time**: live search results, a database table that changes, a countdown, download progress.

	Returns	Example
Normal function	One value, immediately	fun total(): Int
suspend function	One value, later	suspend fun getUser(): User
Flow	Many values, over time	fun observeMessages(): Flow<List<Message>>

2. Creating and collecting a flow

flow { emit() } and collect

```
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*

fun countdown(from: Int): Flow<Int> = flow {
    for (i in from downTo 1) {
        emit(i)              // send a value to the collector
        delay(100)
    }
}

fun main() = runBlocking {
    println("Ready?")
    countdown(3).collect { value ->
        println(value)
    }
    println("Go!")
}
```

Output

```
Ready?
3
2
1
Go!
```

emit(value) sends a value. **collect { }** receives each value. collect is a suspend function: it returns only when the flow finishes.

3. Flows are cold

Nothing happens until you collect

```
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*

fun numbers(): Flow<Int> = flow {
    println("Flow started")
    emit(1)
    emit(2)
}

fun main() = runBlocking {
    val f = numbers()
    println("Flow created, not started yet")
    f.collect { println("Got $it") }
    f.collect { println("Again $it") }      // each collect runs the flow from the start
}
```

Output

```
Flow created, not started yet
Flow started
Got 1
Got 2
Flow started
Again 1
Again 2
```

A normal flow is **cold**: its code runs only when someone collects, and it runs again for every collector. (StateFlow, from Day 5 and Week 8, is **hot**: it always has a value, whether anyone is collecting or not.)

4. Operators: transforming flows

map, filter, take, onEach

```
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*

fun main() = runBlocking {
    val prices = flowOf(120, 450, 80, 999, 300, 60)

    prices
        .filter { it >= 100 }              // keep prices of at least 100
        .map { it * 118 / 100 }            // add 18% tax
        .take(3)                           // only the first 3
        .onEach { println("With tax: Rs $it") }
        .collect()

    val total = prices.filter { it < 500 }.fold(0) { sum, p -> sum + p }
    println("Total under 500: Rs $total")
}
```

Output

```
With tax: Rs 141
With tax: Rs 531
With tax: Rs 1178
Total under 500: Rs 1010
```

Operator	What it does
map { }	Changes each value

Operator	What it does
filter { }	Keeps only matching values
take(n)	Only the first n values, then the flow stops
onEach { }	Does something with each value without changing it (logging, for example)
distinctUntilChanged()	Skips a value if it equals the previous one
toList() / first() / fold()	Terminal operators: collect and return a final result

These look like the list functions from Week 2, but they work on values **as they arrive over time**.

5. Errors and completion in flows

catch and onCompletion

```
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*

fun sensorReadings(): Flow<Int> = flow {
    emit(21)
    emit(22)
    throw RuntimeException("Sensor disconnected")
}

fun main() = runBlocking {
    sensorReadings()
        .catch { e -> println("Error: ${e.message}"); emit(-1) }   // replace the error with a fallback value
        .onCompletion { println("Flow completed") }
        .collect { println("Reading: $it") }
}
```

Output

```
Reading: 21
Reading: 22
Error: Sensor disconnected
Reading: -1
Flow completed
```

6. Where flows come from in Android

Source	Example
Room database (Week 11)	fun observeNotes(): Flow<List<Note>>: emits again whenever the table changes
DataStore (Week 11)	Settings that update live
Your own code	A timer, a location stream, download progress
StateFlow in a ViewModel	The UiState that the screen collects

Practice Exercises

1. Create a flow that emits the numbers 1 to 10 with a 50 ms delay. Collect only the even numbers, squared.

2. Create **fun downloadProgress(): Flow<Int>** that emits 0, 25, 50, 75 and 100. Print "Downloading: N%" for each, and "Download complete" with onCompletion.

3. Use flowOf("a", "a", "b", "b", "a") with distinctUntilChanged() and print the result with toList().

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What is the difference between a suspend function and a Flow?

Q2. What does "cold" mean for a flow?

Q3. What does emit() do?

Q4. Name three flow operators.

Q5. How do you handle an exception thrown inside a flow?

WEEK 9 • DAY 5

StateFlow, combine & debounce

Today's goal: Combine flows, build a live search with debounce, and turn flows into UiState with stateIn.

1. StateFlow and SharedFlow (hot flows)

	StateFlow	SharedFlow
Always has a value?	Yes (one current value)	No
New collectors get	The current value immediately	Only new values (by default)
Repeats equal values?	No	Yes
Use for	**State**: UiState, settings	**Events** that can repeat (some teams use it for messages)

2. combine: build state from several flows

Combining two flows

```
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*

fun main() = runBlocking {
    val items = MutableStateFlow(listOf("Milk", "Bread", "Butter", "Mango"))
    val query = MutableStateFlow("")

    val visible = combine(items, query) { list, q ->
        list.filter { it.contains(q, ignoreCase = true) }
    }

    val job = launch { visible.collect { println("Showing: $it") } }
    delay(100)
    query.value = "b"
    delay(100)
    items.value = items.value + "Biscuits"
    delay(100)
    job.cancel()
}
```

Output

```
Showing: [Milk, Bread, Butter, Mango]
Showing: [Bread, Butter]
Showing: [Bread, Butter, Biscuits]
```

combine recalculates whenever **any** of the input flows changes. It's perfect for screens whose content depends on data plus user choices (search text, filters, sort order).

3. debounce: wait until the user stops typing

Live search with debounce

```
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*

@OptIn(FlowPreview::class)
fun main() = runBlocking {
    val searchText = MutableStateFlow("")

    val job = launch {
        searchText
            .debounce(300)                    // wait for 300 ms of no typing
            .filter { it.length >= 2 }        // ignore very short queries
            .distinctUntilChanged()
            .collect { println("Searching the server for '$it'") }
    }

    for (text in listOf("k", "ko", "kot", "kotl", "kotlin")) {   // user typing fast
        searchText.value = text
        delay(50)
    }
    delay(500)                                // user pauses
    searchText.value = "kotlin flow"
    delay(500)
    job.cancel()
}
```

Output

```
Searching the server for 'kotlin'
Searching the server for 'kotlin flow'
```

Without debounce, every keystroke would call the server (5 requests). With debounce, only **2** searches happen: when the user pauses. This saves data, battery and server costs.

4. stateIn: from Flow to UiState in a ViewModel

A reactive search ViewModel

```
class FruitRepository {
    private val fruits = listOf("Apple", "Banana", "Jackfruit", "Mango", "Papaya", "Pineapple")
    suspend fun search(query: String): List<String> {
        delay(300)                                          // pretend: server
        return fruits.filter { it.contains(query, ignoreCase = true) }
    }
}

data class SearchUiState(val query: String = "", val results: List<String> = emptyList())

@OptIn(FlowPreview::class, ExperimentalCoroutinesApi::class)
class SearchViewModel(private val repo: FruitRepository = FruitRepository()) : ViewModel() {
    private val query = MutableStateFlow("")

    val uiState: StateFlow<SearchUiState> = query
        .debounce(300)
        .mapLatest { q -> SearchUiState(query = q, results = if (q.isBlank()) emptyList() else repo.search(q)) }
        .stateIn(
            scope = viewModelScope,
            started = SharingStarted.WhileSubscribed(5_000),
            initialValue = SearchUiState()
        )

    val queryText: StateFlow<String> = query.asStateFlow()

    fun onQueryChange(text: String) {
        query.value = text
    }
}
```

Code	Meaning
mapLatest { }	Like map, but if a new query arrives while the old search is still running, the old one is cancelled
stateIn(scope, started, initialValue)	Turns a cold Flow into a StateFlow the screen can collect
SharingStarted.WhileSubscribed(5_000)	Keep the flow running while the screen is visible, plus 5 seconds (so rotation doesn't restart it)
queryText	The raw text, shown in the TextField immediately (without the debounce delay)

5. The screen

SearchScreen

```
data class SearchUiState(val query: String = "", val results: List<String> = emptyList())

@Composable
fun SearchScreen(query: String, uiState: SearchUiState, onQueryChange: (String) -> Unit) {
    Column(Modifier.fillMaxSize().padding(16.dp)) {
        OutlinedTextField(
            value = query,
            onValueChange = onQueryChange,
            label = { Text("Search fruits") },
            leadingIcon = { Icon(Icons.Default.Search, contentDescription = null) },
            singleLine = true,
            modifier = Modifier.fillMaxWidth()
        )
        LazyColumn {
            items(uiState.results, key = { it }) { fruit ->
                Text(fruit, modifier = Modifier.fillMaxWidth().padding(16.dp))
            }
        }
    }
}
```

Practice Exercises

1. Combine a flow of products with a flow of a "max price" filter, and print the products under the max price each time the filter changes.

2. Simulate typing "android" one letter every 100 ms with debounce(250). How many searches happen? Run it to check.

3. Add a **sortAscending** MutableStateFlow<Boolean> to the combine example, and sort the visible list based on it.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What does combine do?

Q2. Why use debounce for search?

Q3. What does stateIn do?

Q4. Why WhileSubscribed(5_000)?

Q5. What is the difference between map and mapLatest?

WEEK 9 • DAY 6

Weekly Project: Countdown Timer

Today's goal: Build a countdown timer whose ViewModel runs a coroutine, with start, pause, resume and reset.

1. The design

Sketch: What we're building

```
+------------------------------+
|         .----------.         |
|       /     00:42    \       |   <- CircularProgressIndicator + time
|       \              /       |
|         '----------'         |
|   [00:30] [01:00] [05:00]    |   <- FilterChips choose the duration
|      [ Pause ]  [ Reset ]    |   <- buttons change with the status
+------------------------------+
```

2. The formatting logic

TimeFormat.kt (tested with real output)

```
fun formatTime(totalSeconds: Int): String {
    val minutes = totalSeconds / 60
    val seconds = totalSeconds % 60
    return "%02d:%02d".format(minutes, seconds)
}

fun main() {
    println(formatTime(60))
    println(formatTime(42))
    println(formatTime(305))
    println(formatTime(0))
}
```

Output

```
01:00
00:42
05:05
00:00
```

3. The ViewModel: a coroutine that ticks

TimerViewModel.kt

```
enum class TimerStatus { IDLE, RUNNING, PAUSED, FINISHED }

data class TimerUiState(
    val totalSeconds: Int = 60,
    val remainingSeconds: Int = 60,
    val status: TimerStatus = TimerStatus.IDLE
) {
    val progress: Float get() = if (totalSeconds == 0) 0f else remainingSeconds / totalSeconds.toFloat()
    val display: String get() = formatTime(remainingSeconds)
}

class TimerViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(TimerUiState())
    val uiState: StateFlow<TimerUiState> = _uiState.asStateFlow()

    private var timerJob: Job? = null

    fun setDuration(seconds: Int) {
        if (_uiState.value.status == TimerStatus.RUNNING) return
        _uiState.value = TimerUiState(totalSeconds = seconds, remainingSeconds = seconds)
    }

    fun start() {
        if (timerJob?.isActive == true || _uiState.value.remainingSeconds == 0) return
        _uiState.update { it.copy(status = TimerStatus.RUNNING) }
        timerJob = viewModelScope.launch {
            while (_uiState.value.remainingSeconds > 0) {
                delay(1000)
                _uiState.update { it.copy(remainingSeconds = it.remainingSeconds - 1) }
            }
            _uiState.update { it.copy(status = TimerStatus.FINISHED) }
        }
    }

    fun pause() {
        timerJob?.cancel()
        _uiState.update { it.copy(status = TimerStatus.PAUSED) }
    }

    fun reset() {
        timerJob?.cancel()
        _uiState.update { it.copy(remainingSeconds = it.totalSeconds, status = TimerStatus.IDLE) }
    }
}
```

Concept	Where
A coroutine loop with delay(1000)	start(): while (remaining > 0) { delay(1000); update }
Keeping the Job to cancel it	private var timerJob: Job?; pause() and reset() call timerJob?.cancel()
Avoiding two timers at once	if (timerJob?.isActive == true) return
Automatic cleanup	viewModelScope cancels the timer if the user leaves the screen
An enum for the status	TimerStatus: IDLE, RUNNING, PAUSED, FINISHED (Week 3)
Derived values in UiState	progress and display are computed properties

4. The screen

TimerScreen.kt

```
@Composable
fun TimerRoute(viewModel: TimerViewModel = viewModel()) {
    val uiState by viewModel.uiState.collectAsStateWithLifecycle()
    TimerScreen(
        uiState = uiState,
        onSetDuration = viewModel::setDuration,
        onStart = viewModel::start,
        onPause = viewModel::pause,
        onReset = viewModel::reset
    )
}

@Composable
fun TimerScreen(
    uiState: TimerUiState,
    onSetDuration: (Int) -> Unit,
    onStart: () -> Unit,
    onPause: () -> Unit,
    onReset: () -> Unit
) {
    Column(
        modifier = Modifier.fillMaxSize().padding(24.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Box(contentAlignment = Alignment.Center) {
            CircularProgressIndicator(
                progress = { uiState.progress },
                modifier = Modifier.size(220.dp),
                strokeWidth = 12.dp
            )
            Text(uiState.display, style = MaterialTheme.typography.displayMedium)
        }
        Spacer(Modifier.height(24.dp))

        if (uiState.status == TimerStatus.FINISHED) {
            Text("Time's up!", style = MaterialTheme.typography.headlineSmall,
                color = MaterialTheme.colorScheme.primary)
            Spacer(Modifier.height(16.dp))
        }

        Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
            listOf(30, 60, 300).forEach { seconds ->
                FilterChip(
                    selected = uiState.totalSeconds == seconds,
                    onClick = { onSetDuration(seconds) },
                    enabled = uiState.status != TimerStatus.RUNNING,
                    label = { Text(formatTime(seconds)) }
                )
            }
        }
        Spacer(Modifier.height(16.dp))

        Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
            if (uiState.status == TimerStatus.RUNNING) {
                Button(onClick = onPause) { Text("Pause") }
            } else {
                Button(onClick = onStart, enabled = uiState.remainingSeconds > 0) {
                    Text(if (uiState.status == TimerStatus.PAUSED) "Resume" else "Start")
                }
            }
            OutlinedButton(onClick = onReset) { Text("Reset") }
```

```
        }
    }
}

@Preview(showBackground = true)
@Composable
fun TimerScreenPreview() {
    TimerScreen(TimerUiState(totalSeconds = 60, remainingSeconds = 42, status = TimerStatus.RUNNING),
        { }, { }, { }, { })
}
```

Rotate the phone while the timer is running: it keeps counting, because the coroutine lives in the ViewModel, not in the composable.

Careful: A coroutine timer stops when the ViewModel is cleared, and Android may also stop the app in the background. For alarms that must fire even when the app is closed, use **AlarmManager** or **WorkManager** (Week 13).

5. Project challenges

- Add a custom duration: a text field for minutes that calls setDuration(minutes * 60).
- Make the progress indicator turn red (error colour) in the last 10 seconds.
- Add a **stopwatch** mode that counts up instead of down.
- Make the timer accurate: instead of counting ticks, store the end time with System.currentTimeMillis() and calculate the remaining seconds.

Week 9 checklist

I can...	Done?
Explain the main thread, blocking and ANRs	[ ]
Write suspend functions and start coroutines with launch	[ ]
Run work in parallel with async/await	[ ]
Cancel coroutines and explain structured concurrency	[ ]
Use withContext and the right dispatcher	[ ]
Handle errors, cancellation and timeouts	[ ]
Create, transform and collect flows	[ ]
Use combine, debounce and stateIn in a ViewModel	[ ]
Build the Countdown Timer app	[ ]

Quick Quiz

Q1. Why does the timer keep running when the phone rotates?

Q2. How does pause() stop the timer?

Q3. Why check timerJob?.isActive before starting?

Q4. What happens to the timer coroutine when the user leaves the screen for good?

Q5. Why is formatTime a plain function?

Answer Key

Try every exercise yourself **before** you look here. Your solution may be different and still correct, as long as the output is right!

Day 1: Suspend Functions & launch

Exercise 1

```
import kotlinx.coroutines.*

suspend fun makeTea(): String {
    println("Boiling water")
    delay(300)
    println("Adding tea")
    delay(200)
    return "Tea is ready"
}

fun main() = runBlocking {
    println(makeTea())
}
```

Output

```
Boiling water
Adding tea
Tea is ready
```

Exercise 2

```
import kotlinx.coroutines.*

fun main() = runBlocking {
    launch { delay(300); println("A (300 ms)") }
    launch { delay(100); println("B (100 ms)") }
    launch { delay(200); println("C (200 ms)") }
    println("All three started")
}
```

Output

```
All three started
B (100 ms)
C (200 ms)
A (300 ms)
```

Exercise 3

```
import kotlinx.coroutines.*

// With Thread.sleep, each coroutine blocks the single thread, so they finish
// in the order they started: A, B, C (and it takes 600 ms instead of 300 ms).
fun main() = runBlocking {
    launch { Thread.sleep(300); println("A") }
    launch { Thread.sleep(100); println("B") }
    launch { Thread.sleep(200); println("C") }
    println("All three started")
}
```

Output

```
All three started
A
B
C
```

Quiz answers

Q1. Android shows an "App isn't responding" (ANR) dialog.

Q2. The function can pause without blocking the thread; it must be called from a coroutine or another suspend function.

Q3. delay suspends only the coroutine; Thread.sleep blocks the whole thread.

Q4. No. It's for learning, main() and tests only.

Q5. viewModelScope

Day 2: async, Jobs & Structured Concurrency

Exercise 1

```
import kotlinx.coroutines.*
import kotlin.system.measureTimeMillis

suspend fun getWeather(): String { delay(400); return "Sunny" }
suspend fun getNews(): String { delay(300); return "Kerala wins" }

fun main() = runBlocking {
    val time = measureTimeMillis {
        val w = async { getWeather() }
        val n = async { getNews() }
        println("${w.await()} / ${n.await()}")
    }
    println("About ${time / 100 * 100} ms")
}
```

Output

```
Sunny / Kerala wins
About 400 ms
```

Exercise 2

```
import kotlinx.coroutines.*

fun main() = runBlocking {
    val job = launch {
        for (i in 10 downTo 1) {
            println(i)
            delay(100)
        }
    }
    delay(450)
    job.cancel()
    println("Cancelled")
}
```

Output

```
10
9
8
7
6
Cancelled
```

Exercise 3

```
import kotlinx.coroutines.*

suspend fun loadAll(ids: List<Int>): List<String> = coroutineScope {
    ids.map { id -> async { delay(100); "Item $id" } }.awaitAll()
}

fun main() = runBlocking {
    println(loadAll(listOf(1, 2, 3)))
}
```

Output

```
[Item 1, Item 2, Item 3]
```

Quiz answers

Q1. launch returns a Job with no result; async returns a Deferred whose result you get with await().

Q2. Both calls run at the same time.

Q3. At its next suspension point, such as delay: cancellation is cooperative.

Q4. They are cancelled too.

Q5. Coroutines form a parent-child tree, so none are forgotten and cancellation spreads to children.

Day 3: Dispatchers, Scopes & Errors

Exercise 1

```
import kotlinx.coroutines.*

suspend fun sumOfSquares(n: Int): Long = withContext(Dispatchers.Default) {
    (1..n).sumOf { it.toLong() * it }
}

fun main() = runBlocking {
    println(sumOfSquares(1_000_000))
}
```

Output

```
333333833333500000
```

Exercise 2

```
import kotlinx.coroutines.*

suspend fun squareRootOf(n: Int): Double {
    delay(50)
    require(n >= 0) { "Negative number: $n" }
    return kotlin.math.sqrt(n.toDouble())
}

fun main() = runBlocking {
    println(runCatching { squareRootOf(25) })
    println(runCatching { squareRootOf(-5) }.exceptionOrNull()?.message)
}
```

Output

```
Success(5.0)
Negative number: -5
```

Exercise 3

```
import kotlinx.coroutines.*

suspend fun task(ms: Long): String { delay(ms); return "done in $ms ms" }

fun main() = runBlocking {
    println(withTimeoutOrNull(300) { task(500) })
    println(withTimeoutOrNull(300) { task(100) })
}
```

Output

```
null
done in 100 ms
```

Quiz answers

Q1. Dispatchers.IO

Q2. Dispatchers.Default

Q3. Runs a block on another dispatcher, waits for it, and returns its result.

Q4. It also catches CancellationException, which can stop cancellation from working.

Q5. withTimeout throws when time runs out; withTimeoutOrNull returns null.

Day 4: Flow Basics

Exercise 1

```
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*

fun main() = runBlocking {
    flow { for (i in 1..10) { emit(i); delay(50) } }
        .filter { it % 2 == 0 }
        .map { it * it }
        .collect { println(it) }
}
```

Output

```
4
16
36
64
100
```

Exercise 2

```
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*

fun downloadProgress(): Flow<Int> = flow {
    for (p in 0..100 step 25) { emit(p); delay(50) }
}

fun main() = runBlocking {
    downloadProgress()
        .onCompletion { println("Download complete") }
        .collect { println("Downloading: $it%") }
}
```

Output

```
Downloading: 0%
Downloading: 25%
Downloading: 50%
Downloading: 75%
Downloading: 100%
Download complete
```

Exercise 3

```
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*

fun main() = runBlocking {
    println(flowOf("a", "a", "b", "b", "a").distinctUntilChanged().toList())
}
```

Output

```
[a, b, a]
```

Quiz answers

Q1. A suspend function returns one value; a Flow emits many values over time.

Q2. Its code runs only when collected, and again for each collector.

Q3. Sends a value to whoever is collecting the flow.

Q4. map, filter, take (also onEach, distinctUntilChanged, catch...).

Q5. With the catch { } operator (or try/catch around collect).

Day 5: StateFlow, combine & debounce

Exercise 1

```
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*

data class Product(val name: String, val price: Int)

fun main() = runBlocking {
    val products = MutableStateFlow(listOf(Product("Pen", 10), Product("Bag", 900), Product("Book", 350)))
    val maxPrice = MutableStateFlow(1000)
    val job = launch {
        combine(products, maxPrice) { list, max -> list.filter { it.price <= max }.map { it.name } }
            .collect { println("Under limit: $it") }
    }
    delay(100); maxPrice.value = 400
    delay(100); maxPrice.value = 50
    delay(100); job.cancel()
}
```

Output

```
Under limit: [Pen, Bag, Book]
Under limit: [Pen, Book]
Under limit: [Pen]
```

Exercise 2

```
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*

@OptIn(FlowPreview::class)
fun main() = runBlocking {
    val text = MutableStateFlow("")
    var searches = 0
    val job = launch { text.debounce(250).filter { it.isNotEmpty() }.collect { searches++; println("Search: $it") } }
    for (i in 1.."android".length) { text.value = "android".take(i); delay(100) }
    delay(400)
    job.cancel()
    println("Searches: $searches")
}
```

Output

```
Search: android
Searches: 1
```

Exercise 3

```
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*

fun main() = runBlocking {
    val items = MutableStateFlow(listOf("Mango", "Apple", "Banana"))
    val ascending = MutableStateFlow(true)
    val job = launch {
        combine(items, ascending) { list, asc -> if (asc) list.sorted() else list.sortedDescending() }
            .collect { println(it) }
    }
    delay(100); ascending.value = false
    delay(100); job.cancel()
}
```

Output

```
[Apple, Banana, Mango]
[Mango, Banana, Apple]
```

Quiz answers

Q1. Produces a new value from several flows whenever any of them changes.

Q2. To wait until the user stops typing, so the server isn't called on every keystroke.

Q3. Converts a Flow into a StateFlow with an initial value, shared in a scope.

Q4. It keeps the flow alive briefly when the screen stops collecting, so rotation doesn't restart the work.

Q5. mapLatest cancels the previous transformation when a new value arrives.

Day 6: Weekly Project: Countdown Timer

Quiz answers

Q1. The coroutine runs in viewModelScope, and the ViewModel survives rotation.

Q2. It cancels the saved Job; the delay in the loop is a cancellation point.

Q3. To avoid running two timers at the same time.

Q4. viewModelScope is cancelled, so the timer stops automatically.

Q5. So the logic can be tested without Android, as in the example with real output.

Next: When you finish this week, tell your mentor and ask for the **Week 10** material. Great work, Anjana!